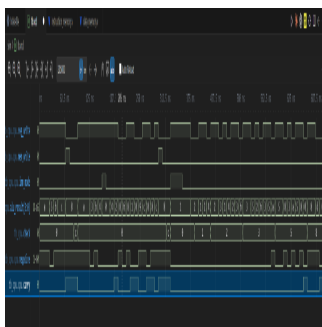


ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA VẬT LÝ - VẬT LÝ KỸ THUẬT
BỘ MÔN VẬT LÝ TIN HỌC / THIẾT KẾ VI MẠCH



BÁO CÁO ĐỒ ÁN MÔN HỌC
THIẾT KẾ BỘ VI XỬ LÝ CPU 4-BIT
(TKVM)

Hệ thống xử lý đơn chu kỳ (Single-Cycle) trên RTL & Layout vật lý

Sinh viên thực hiện: Lê Ngọc Tường

Mã số sinh viên: 22120395

Lớp: Thiết kế vi mạch điện tử - HK3/2025-2026

Giáo viên hướng dẫn: Ban Giảng huấn Bộ môn Vi mạch

Hệ thống thiết kế: Kiến trúc Harvard 4-bit Single-Cycle

Thành phố Hồ Chí Minh, Tháng 7 năm 2026

TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN - ĐHQG-HCM

KHOA VẬT LÝ - VẬT LÝ KỸ THUẬT

BỘ MÔN VẬT LÝ TIN HỌC / THIẾT KẾ VI MẠCH

BÁO CÁO ĐỒ ÁN MÔN HỌC: THIẾT KẾ VI MẠCH ĐIỆN TỬ

ĐỀ TÀI: THIẾT KẾ VÀ KIỂM THỬ BỘ VI XỬ LÝ CPU 4-BIT ĐƠN CHU KỲ (TKVM)

KIẾN TRÚC HARVARD — RTL SIMULATION & PHYSICAL LAYOUT (GDSII)

- Sinh viên thực hiện: Lê Ngọc Tường
- Mã số sinh viên: 22120395
- Lớp: Thiết kế vi mạch điện tử - Học kỳ 3 / 2025-2026
- Giáo viên hướng dẫn: Ban Giảng huấn Bộ môn Vi mạch

MỤC LỤC

- Chương 1: Tổng quan kiến trúc hệ thống
- Chương 2: Kiến trúc tập lệnh (ISA) & Bộ điều khiển
- Chương 3: Thiết kế phần cứng cấp RTL

4. Chương 4: Mô phỏng RTL & Kiểm thử chức năng

Báo cáo đồ án: Thiết kế bộ vi xử lý CPU 4-bit (TKN)

5. Chương 5: Logic Synthesis & Thống kê tài nguyên
6. Chương 6: Báo cáo định thời & Phân tích đường trễ tới hạn
7. Chương 7: Tích hợp cơ chế chốt cờ điều khiển (flag_write)
8. Chương 8: Kiểm chứng chức năng & Thực nghiệm
9. Kết luận

DANH MỤC HÌNH VẼ

- Hình 1: Sơ đồ khối tổng quan kiến trúc CPU 4-bit (Harvard Architecture)
- Hình 2: Dạng sóng mô phỏng tổng quan chạy chuỗi 3 chương trình (waveform_overview)
- Hình 3: Dạng sóng hoạt động của tập thanh ghi R0 - R3 (waveform_registers)
- Hình 4: Dạng sóng các tín hiệu điều khiển và kết quả ALU (waveform_control)
- Hình 5: Dạng sóng chi tiết thực thi phép nhân $3 \times 4 = 12$ (waveformcputop)
- Hình 6: Sơ đồ netlist mức cổng (Gate-level Netlist) toàn CPU do Yosys tổng hợp
- Hình 7: Sơ đồ netlist mức cổng chi tiết của khối tính toán ALU 4-bit
- Hình 8: Layout vật lý GDSII của CPU hiển thị trên KLayout sau quy trình OpenLane
- Hình 9: Dạng sóng mô phỏng kiểm thử cờ có điều kiện (waveform_flag)

DANH MỤC BẢNG BIỂU

- Bảng 1: Bảng mô tả ngữ nghĩa tập lệnh (ISA Semantics)
- Bảng 2: Bảng chân trị giải mã tín hiệu điều khiển tích hợp flag_write
- Bảng 3: Phạm vi kiểm thử và phân tầng của các tệp VCD mô phỏng
- Bảng 4: Bảng trace hoạt động chu kỳ thực thi chương trình nhân (MUL)
- Bảng 5: Kịch bản kiểm thử tối thiểu và kết quả kiểm chứng chức năng

Chương 1: Tổng quan kiến trúc hệ thống

Báo cáo đồ án: Thiết kế bộ vi xử lý CPU 4-bit (TKN)

Dự án này hiện thực hóa một bộ vi xử lý (CPU) 4-bit đơn chu kỳ (single-cycle) theo kiến trúc Harvard phục vụ môn học Thiết kế vi mạch điện tử. Thiết kế này phân tách độc lập giữa bộ nhớ chứa chương trình (ROM) và bộ nhớ chứa dữ liệu (RAM) để đạt hiệu năng xử lý cao, tránh nghẽn cổ chai bus dữ liệu truyền thống.

1.1. Thông số kỹ thuật cốt lõi

- Độ rộng từ lệnh (Instruction Word): 9 bit. Cấu trúc lệnh bao gồm bit chế độ hằng số `imm_mode` [8], mã lệnh `opcode` [7:4], và toán hạng `operand` [3:0].
- Độ rộng bus dữ liệu (Data Bus): 4 bit.
- Tập thanh ghi (Register File): Gồm 4 thanh ghi đa dụng từ R0 đến R3 (4 bit mỗi thanh ghi). Trong đó, thanh ghi R0 đóng vai trò thanh ghi tích lũy (accumulator) ngầm định cho các lệnh truy cập bộ nhớ (`LOAD`, `STORE`) và nạp hằng số (`LDI`).
- Bộ nhớ chỉ lệnh (Instruction ROM): Kích thước 16 từ lệnh $\times$ 9 bit.
- Bộ nhớ dữ liệu (Data RAM): Kích thước 16 ô nhớ $\times$ 4 bit.
- Thanh ghi trạng thái (Flags Register): Lấy mẫu và chốt 2 cờ trạng thái gồm cờ Zero (Z), cờ âm Negative (N) tại cạnh lên của xung nhịp (clock edge). Các cờ này được chốt có điều kiện nhờ tín hiệu điều khiển `flag_write` để đảm bảo lệnh nhảy điều kiện đọc đúng trạng thái từ phép toán arithmetic/logic liền trước mà không bị ghi đè bởi các lệnh không liên quan.
- Chu kỳ thực thi: Đơn chu kỳ (Single-cycle), mỗi lệnh hoàn thành trong đúng một chu kỳ xung nhịp.
- Tín hiệu điều khiển đặc biệt: `HALT` (khi tích cực sẽ khóa bộ đếm chương trình PC để dừng hệ thống).

Chương 2: Kiến trúc tập lệnh (ISA) & Bộ điều khiển

Bộ vi xử lý hoạt động dựa trên định dạng từ lệnh có độ rộng cố định là 9 bit. Cấu trúc từ lệnh được mô tả như sau:

8	7	4	3	0	+	-----	+	-----	+	-----	+		imm_mode		opcode		operand			(1 bit)
						(4 bit)			(4 bit)		+	-----	+							

- `imm_mode` (bit [8]): Chỉ có ý nghĩa phân biệt chế độ hoạt động khi `opcode` = `0011` (giá trị 1 chọn lệnh nạp hằng số `LDI`, giá trị 0 chọn lệnh sao chép thanh ghi `MOV`). Đối với các opcode khác, cách diễn giải operand được quyết định bởi opcode; `imm_mode` là bit không sử dụng và được assembler mặc định gán về 0.
- `opcode` (bit [7:4]): Có 16 giá trị mã hóa từ `0000` đến `1111`. Riêng opcode `0011` kết hợp với `imm_mode` để phân biệt lệnh `LDI` và `MOV`.
- `operand` (bit [3:0]): Toán hạng phụ thuộc vào từng lệnh.

2.1. Giải pháp mã hóa lệnh bằng tổ hợp `{imm_mode, opcode}`

Để tối ưu hóa không gian mã hóa opcode 4-bit, thiết kế sử dụng chung mã opcode **0011** cho hai lệnh **LDI** và **MOV**. Bộ giải mã điều khiển giải quyết trùng chấp bằng cách kết hợp cả bit **imm_mode**:

- Khi **{imm_mode, opcode} = 5'b1_0011**: Giải mã là lệnh **LDI**, toán hạng 4-bit **operand[3:0]** được đưa qua ALU và ghi vào thanh ghi tích lũy **\$R_0\$**.
- Khi **{imm_mode, opcode} = 5'b0_0011**: Giải mã là lệnh **MOV**, toán hạng được chia thành thanh ghi đích **\$Rd\$** (**operand[3:2]**) và thanh ghi nguồn **\$Rs\$** (**operand[1:0]**).

2.2. Bảng mô tả ngữ nghĩa tập lệnh (ISA Semantics)

Dưới đây là đặc tả ngữ nghĩa hoạt động của các lệnh cùng hành vi cờ tương ứng đối với phiên bản phần cứng hiện tại (khi cờ trạng thái được chốt có điều kiện qua **flag_write** ở mỗi chu kỳ clock):

Bảng 1: Bảng mô tả ngữ nghĩa tập lệnh (ISA Semantics)

Opcode	Lệnh	Định dạng ASM	Phép toán mức RTL (Register Transfer Level)	Hành vi cờ của RTL hiện tại
0000	NOP	NOP	Không thao tác	Giữ nguyên trạng thái cờ trước đó
0001	LOAD	LOAD addr	$\$R_0 \leftarrow \text{RAM}[\text{addr}]$	Giữ nguyên trạng thái cờ trước đó
0010	STORE	STORE addr	$\text{RAM}[\text{addr}] \leftarrow R_0$	Giữ nguyên trạng thái cờ trước đó
0011	MOV	MOV Rd, Rs	$\$R[Rd] \leftarrow R[Rs]$	Giữ nguyên trạng thái cờ trước đó
0011	LDI	LDI #imm	$\$R_0 \leftarrow \text{imm_val}$	$\$Z, \N phản ánh giá trị hằng số nạp
0100	ADD	ADD Rd, Rs	$\$R[Rd] \leftarrow R[Rd] + R[R_s]$	$\$Z, \N phản ánh kết quả phép cộng
0101	SUB	SUB Rd, Rs	$\$R[Rd] \leftarrow R[Rd] - R[R_s]$	$\$Z, \N phản ánh kết quả phép trừ
0110	AND	AND Rd, Rs	$\$R[Rd] \leftarrow R[Rd] \& R[R_s]$	$\$Z, \N phản ánh kết quả phép AND
0111	OR	OR Rd, Rs	$\$R[Rd] \leftarrow R[Rd] \vee R[R_s]$	$\$Z, \N phản ánh kết quả phép OR
1000	XOR	XOR Rd, Rs	$\$R[Rd] \leftarrow R[Rd] \oplus R[R_s]$	$\$Z, \N phản ánh kết quả phép XOR
1001	INC	INC Rd	$\$R[Rd] \leftarrow R[Rd] + 1$	$\$Z, \N phản ánh kết quả phép tăng
1010	DEC	DEC Rd	$\$R[Rd] \leftarrow R[Rd] - 1$	$\$Z, \N phản ánh kết quả phép giảm
1011	JMP	JMP addr	$\text{PC} \leftarrow \text{addr}$	Giữ nguyên trạng thái cờ trước đó
1100	JZ	JZ addr	$\text{if } Z = 1: \text{PC} \leftarrow \text{addr}$	Giữ nguyên trạng thái cờ trước đó
1101	JN	JN addr	$\text{if } N = 1: \text{PC} \leftarrow \text{addr}$	Giữ nguyên trạng thái cờ trước đó
1110	OUT	OUT Rs	$\text{OUT} \leftarrow R[R_s]$	Giữ nguyên trạng thái cờ trước đó

> [!NOTE]

> Giải thích tính bất đối xứng về hành vi cờ giữa LDI và LOAD:

> Lệnh **LDI #imm** thực hiện cập nhật cờ trạng thái (\$Z, N\$), trong khi lệnh **LOAD addr** (đọc dữ liệu từ RAM ghi vào R0) thì giữ nguyên trạng thái cờ trước đó. Đây là một quyết định thiết kế kiến trúc có chủ đích để tối ưu hóa phần cứng:

> 1. Tối ưu hóa đường trễ tới hạn (Critical Path): Thao tác đọc dữ liệu từ RAM của lệnh **LOAD** là mạch tổ hợp bất đồng bộ diễn ra ở nửa sau chu kỳ xung nhịp và có độ trễ truy xuất lớn. Nếu cho phép dữ liệu đọc từ RAM đi qua khối ALU để sinh cờ trạng thái và chốt lại tại cạnh lên clock, đường trễ tới hạn sẽ bị kéo dài cực hạn (bao gồm trễ đọc RAM + trễ ALU + trễ setup cờ). Điều này sẽ làm giảm đáng kể tần số hoạt động cực đại f_{max} của toàn CPU.

> 2. Giải pháp phân tách: Việc không cập nhật cờ khi **LOAD** giúp rút ngắn critical path tối đa. Nếu lập trình viên muốn kiểm tra trạng thái của dữ liệu vừa nạp từ RAM, họ có thể sử dụng một lệnh ALU bổ sung (như **AND R0, R0** hoặc **ADD R0, #0**) để cập nhật cờ một cách an toàn mà không làm ảnh hưởng đến tần số nhịp hệ thống.

2.3. Bảng chân trị giải mã tín hiệu (Control Decoding)

Dựa vào mã opcode và bit **imm_mode**, Control Unit phát ra các tín hiệu điều khiển tương ứng để cấu hình ALU, các bộ dồn kênh (Multiplexer), và kích hoạt cập nhật cờ trạng thái thông qua tín hiệu **flag_write**:

Bảng 2: Bảng chân trị giải mã tín hiệu điều khiển tích hợp flag_write

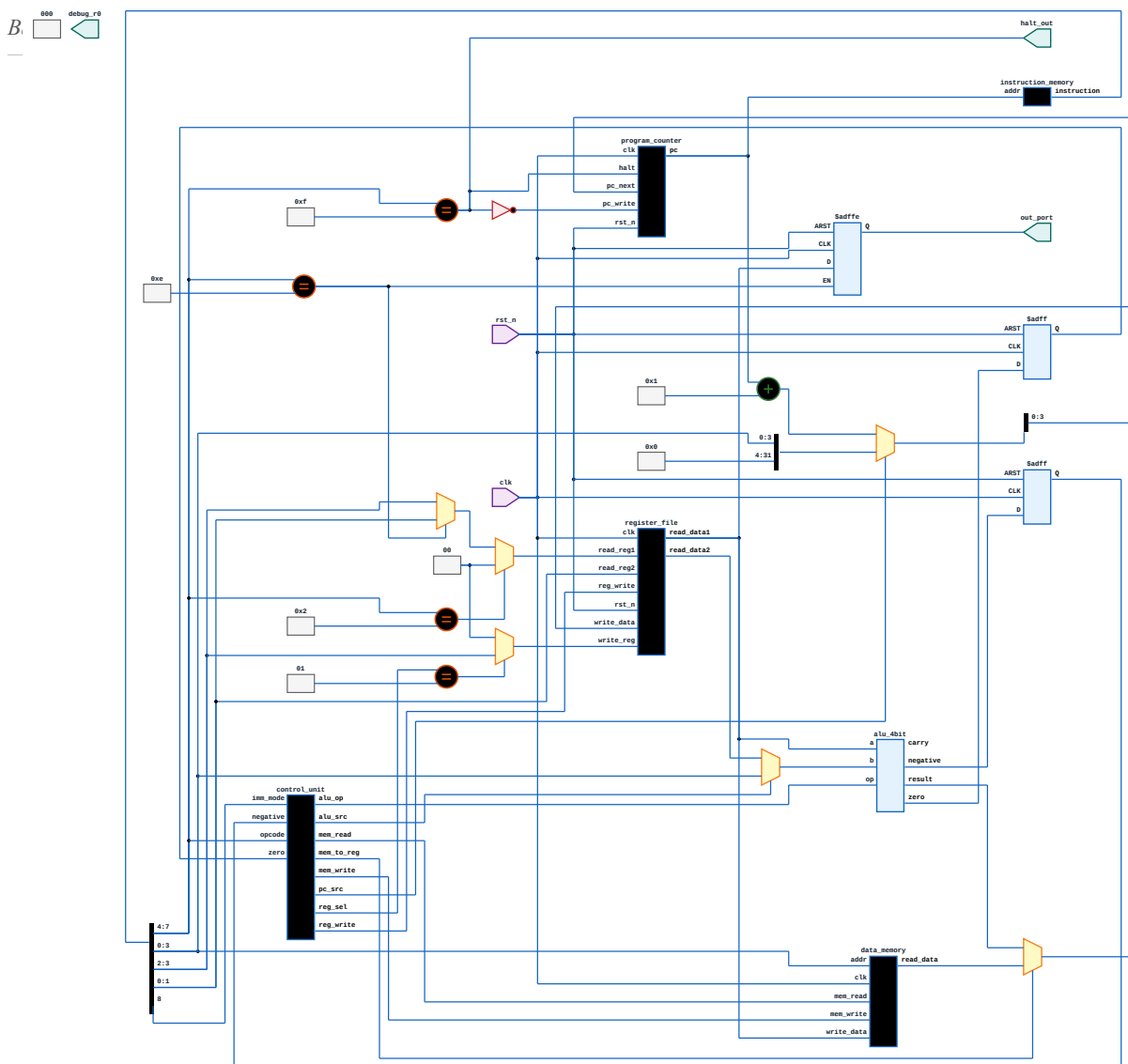
Opcode	Lệnh	immmode	RegWrite	MemRead	MemWrite	MemToReg	ALUSrc	ALUOp	PCSrc	flagwrite	Ghi chú
:---	:---	:---	:---	:---	:---	:---	:---	:---	:---	:---	
0000	NOP	X	0	0	0	0	0	000	0	0	Không thao tác
0001	LOAD	X	1	1	0	1	1	000	0	0	Nạp R0 = RAM[addr]
0010	STORE	X	0	0	1	X	1	000	0	0	Ghi RAM[addr] = R0
0011	LDI	1	1	0	0	0	1	111	0	1	Nạp R0 = imm
0011	MOV	0	1	0	0	0	0	111	0	0	Sao chép Rd = Rs
0100	ADD	0	1	0	0	0	0	000	0	1	Cộng Rd = Rd + Rs
0101	SUB	0	1	0	0	0	0	001	0	1	Trừ Rd = Rd - Rs
0110	AND	0	1	0	0	0	0	010	0	1	Logic AND Rd = Rd & Rs
0111	OR	0	1	0	0	0	0	011	0	1	Logic OR Rd = Rd Rs
1000	XOR	0	1	0	0	0	0	100	0	1	Logic XOR Rd = Rd ^ Rs
1001	INC	0	1	0	0	0	0	101	0	1	Tăng Rd = Rd + 1

Bảng 1.10: Bảng mã lệnh của CPU 4-bit (TK1114)

1010	DEC	X	0	0	0	0	0	1	0	1	Giảm Rd = Rd - 1
1011	JMP	X	0	0	0	X	X	xxx	1	0	Nhảy PC = addr
1100	JZ	X	0	0	0	X	X	xxx	Zr	0	Nhảy nếu cờ Zr = 1
1101	JN	X	0	0	0	X	X	xxx	Nr	0	Nhảy nếu cờ Nr = 1
1110	OUT	X	0	0	0	X	X	000	0	0	Xuất out_port = Rs
1111	HALT	X	0	0	0	X	X	xxx	0	0	Dừng PC, kết thúc

Chương 3: Thiết kế phần cứng cấp RTL

Sơ đồ khối của CPU 4-bit được mô tả chi tiết trong Hình 1, thể hiện kết nối giữa các khối chức năng.



Hình: Hình 1: Sơ đồ khối tổng quan kiến trúc CPU 4-bit

3.1. Các khối chức năng chính:

1. Bộ đếm chương trình (Program Counter - PC): Quản lý địa chỉ lệnh hiện tại (4 bit). PC cập nhật giá trị PC + 1 tại mỗi chu kỳ xung nhịp hoặc nạp địa chỉ mới từ toán hạng lệnh nếu có tín hiệu nhảy (**JMP**, **JZ**, **JN**) kích cực.
2. Bộ nhớ chỉ lệnh (Instruction Memory): Bộ nhớ ROM 16×9-bit, nhận địa chỉ từ PC để xuất ra từ lệnh rộng 9 bit tương ứng.
3. Bộ điều khiển (Control Unit): Đọc opcode và bit chế độ từ lệnh để sinh các tín hiệu điều khiển hệ thống (**reg_write**, **mem_write**, **mem_to_reg**, **alu_op**, **alu_src**, **flag_write**).

4. Tập thanh ghi (Register File): Chứa 4 thanh ghi đa dụng R0-R3. Cho phép đọc đồng thời 2 thanh ghi và ghi 1 thanh ghi tại cạnh lên xung nhịp khi tín hiệu `reg_write` tích cực.

5. Khối tính toán Số học & Logic (ALU): Thực hiện 8 chức năng tính toán chính trên dữ liệu 4 bit (Cộng, Trừ, AND, OR, XOR, Tăng, Giảm, Pass-through). Kết quả ALU cập nhật trạng thái các cờ trạng thái (Z, N, C) để hồi tiếp về khối điều khiển PC.

6. Bộ nhớ dữ liệu (Data Memory): Bộ nhớ RAM 16×4-bit dùng để lưu trữ dữ liệu tính toán, kích hoạt đọc ghi thông qua tín hiệu `mem_read` và `mem_write`.

3.2. Mô tả RTL phần cứng (SystemVerilog)

3.2.1. Khối Số học và Logic (ALU)

Khối ALU thực hiện tính toán số học trên dữ liệu 4-bit. Để đảm bảo tính độc lập với quy tắc tự xác định độ rộng của trình biên dịch, các toán hạng cộng và trừ được mở rộng rõ ràng lên 5-bit trước khi thực hiện để trích xuất cờ nhớ `carry` tổ hợp.

```
always_comb begin result = 4'b0000; carry = 1'b0; case (op) 3'b000: begin // ADD {carry, result} = {1'b0, a} + {1'b0, b}; end 3'b001: begin // SUB (a - b) // carry = 1: Không xảy ra mượn (a >= b) // carry = 0: Có mượn (a < b) {carry, result} = {1'b0, a} + {1'b0, ~b} + 5'b00001; end 3'b010: result = a & b; // AND 3'b011: result = a | b; // OR 3'b100: result = a ^ b; // XOR 3'b101: begin // INC {carry, result} = {1'b0, a} + 5'b00001; end 3'b110: begin // DEC (a - 1) {carry, result} = {1'b0, a} + {1'b0, 4'b1111}; end 3'b111: result = b; // PASSTHRU (Dùng cho LDI / MOV) default: begin result = 4'b0000; carry = 1'b0; end endcase end assign zero = (result == 4'b0000); assign negative = result[3]; // Bit MSB của số có dấu bù 2
```

3.2.2. Khối điều khiển (Control Unit)

Bộ điều khiển giải mã mã lệnh để phát tín hiệu. Khối sử dụng cấu trúc `always_comb` để đảm bảo tính chất mạch tổ hợp và giải mã thêm cổng ra điều khiển `flag_write`:

```
module control_unit ( input logic [3:0] opcode, input logic zero, input logic negative, input logic imm_mode, output logic reg_write, output logic mem_read, output logic mem_write, output logic mem_to_reg, output logic alu_src, output logic [2:0] alu_op, output logic pc_src, output logic [1:0] reg_sel, output logic flag_write ); always @(opcode, imm_mode) begin // defaults reg_write = 1'b0; mem_read = 1'b0; mem_write = 1'b0; mem_to_reg = 1'b0; alu_src = 1'b0; alu_op = 3'b000; reg_sel = 2'b00; flag_write = 1'b0; case (opcode) 4'b0000: alu_op = 3'b000; // NOP 4'b0001: begin // LOAD reg_write = 1'b1; mem_read = 1'b1; mem_to_reg = 1'b1; alu_src = 1'b1; reg_sel = 2'b00; end 4'b0010: begin // STORE mem_write = 1'b1; alu_src = 1'b1; end 4'b0011: begin // LDI / MOV reg_write = 1'b1; mem_to_reg = 1'b0; alu_src = imm_mode; alu_op = 3'b111; reg_sel = imm_mode ? 2'b00 : 2'b01; flag_write = imm_mode; // Chỉ LDI cập nhật cờ end 4'b0100: begin // ADD reg_write = 1'b1; alu_op = 3'b000; reg_sel = 2'b01; flag_write = 1'b1; end 4'b0101: begin // SUB reg_write = 1'b1; alu_op = 3'b001; reg_sel = 2'b01; flag_write = 1'b1; end 4'b0110: begin // AND reg_write = 1'b1; alu_op = 3'b010; reg_sel = 2'b01; flag_write = 1'b1; end 4'b0111: begin // OR reg_write = 1'b1; alu_op = 3'b011; reg_sel = 2'b01; flag_write = 1'b1; end 4'b1000: begin // XOR reg_write = 1'b1; alu_op = 3'b100; reg_sel = 2'b01; flag_write = 1'b1; end 4'b1001: begin // INC reg_write = 1'b1; alu_op = 3'b101; reg_sel = 2'b01; flag_write = 1'b1; end 4'b1010: begin // DEC reg_write = 1'b1; alu_op = 3'b110; reg_sel = 2'b01; flag_write = 1'b1; end 4'b1110: alu_op = 3'b000; // OUT default: alu_op = 3'b000; endcase end assign pc_src = (opcode == 4'b1011) ? 1'b1 : (opcode == 4'b1100) ? zero : (opcode == 4'b1101) ? negative : 1'b0; endmodule
```

3.2.3. Khối chốt cờ trạng thái tích hợp flag_write

Để giải phóng ràng buộc phần mềm bắt buộc lệnh nhảy điều kiện phải đứng ngay sau lệnh sinh cờ, CPU tích hợp khối chốt cờ trạng thái có điều kiện trong `cpu_top.v`. Các cờ Zero (**zero_r**) và Negative (**negative_r**) chỉ được cập nhật khi tín hiệu **flag_write = 1** được phát ra bởi bộ giải mã Control Unit:

```
always_ff @(posedge clk or negedge rst_n) begin if (!rst_n) begin zero_r <= 1'b0; negative_r <= 1'b0; end else if (flag_write) begin zero_r <= zero; negative_r <= negative; end end
```

Chương 4: Mô phỏng RTL & Kiểm thử chức năng

Quá trình kiểm thử chức năng được phân tầng rõ rệt thông qua hai kịch bản testbench độc lập để tạo ra chuỗi bằng chứng kỹ thuật vững chắc:

Bảng 3: Phạm vi kiểm thử và phân tầng của các tệp VCD mô phỏng

STT	Tệp VCD kết quả	Testbench sinh	Đối tượng kiểm thử	Nội dung kiểm thử	Phương pháp kiểm thử
1	<code>sim/tb.vcd</code>	<code>tb/tb_alu_4bit.v</code>	ALU đơn lẻ (<code>alu_4bit</code>)	Kiểm thử toàn bộ 8 phép toán số học và logic, cờ trạng thái cận biên.	Kiểm thử hộp trắng (White-box testing), kích tín hiệu ngõ vào trực tiếp.
2	<code>cpu_top.vcd</code>	<code>tb/tb_cpu_top.v</code>	Hệ thống CPU Top (<code>cpu_top</code>)	Chạy nối tiếp 3 chương trình thực tế: MUL (Phép nhân) -> ADD (Phép cộng) -> FIB (Dãy Fibonacci).	Kiểm thử hộp đen (Black-box testing), nạp mã hex vào ROM và chạy.

4.1. Kết quả log kiểm thử hệ thống

Kết quả chạy tự động được ghi nhận thành công từ tệp log đầu ra:

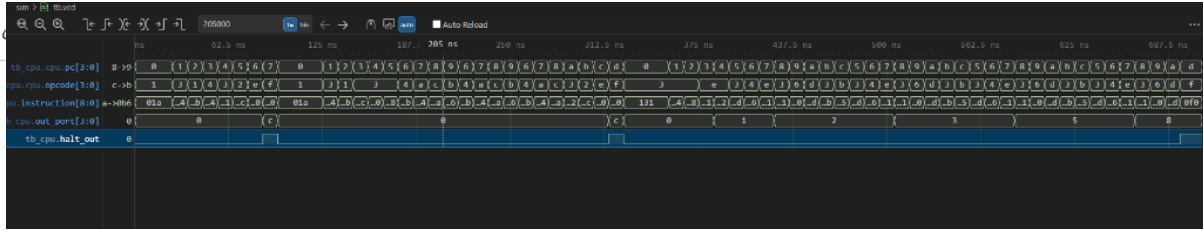
```
TB INITIAL START VCD info: dumpfile cpu_top.vcd opened for output. ---- MUL ---- [PASS] MUL : halt after 21 cycles, out_port=12 (0xc) ---- ADD ---- [PASS] ADD : halt after 8 cycles, out_port=12 (0xc) ---- FIB ---- [PASS] FIB : halt after 36 cycles, out_port=8 (0x8) FIB OUT sequence captured (6 values): 1, 1, 2, 3, 5, 8 ===== RESULT: 3 PASS, 0 FAIL ===== ALL TESTS PASSED
```

4.2. Phân tích các biểu đồ dạng sóng mô phỏng

Dưới đây là các ảnh chụp dạng sóng mô phỏng được phân tích chi tiết từ GTKWave:

4.2.1. Dạng sóng tổng quan (Overview Waveform)

Hình 2 chụp lại chuỗi chuyển đổi giữa 3 chương trình kiểm thử.

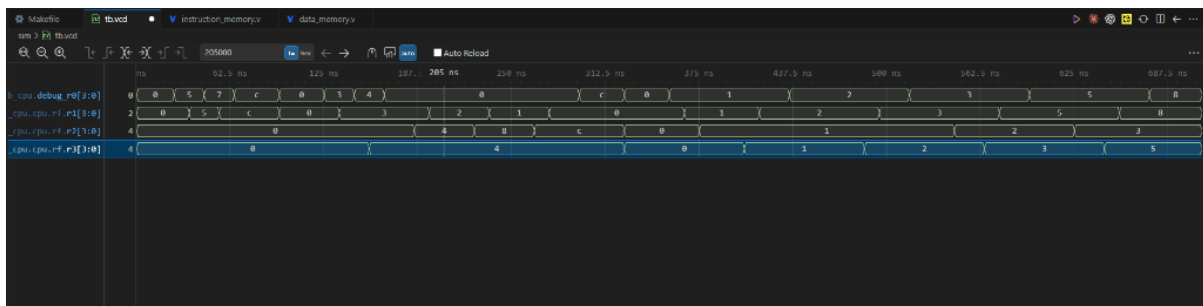


Hình: Hình 2: Dạng sóng mô phỏng tổng quan chuỗi chương trình

- Phân tích: Xung nhịp **clk** dao động với chu kỳ 10ns. Khi giải phóng reset (**rst_n = 1**), bộ đếm **pc** tăng tuần tự, đọc các mã lệnh từ ROM thông qua bus **instruction**. Kết quả đầu ra của cổng hiển thị chính xác tại **out_port** và tín hiệu dừng **halt_out** nhảy lên mức cao mỗi khi một chương trình kết thúc và chuyển tiếp.

4.2.2. Dạng sóng tập thanh ghi (Registers Waveform)

Hình 3 thể hiện quá trình cập nhật dữ liệu của tập thanh ghi R0-R3.

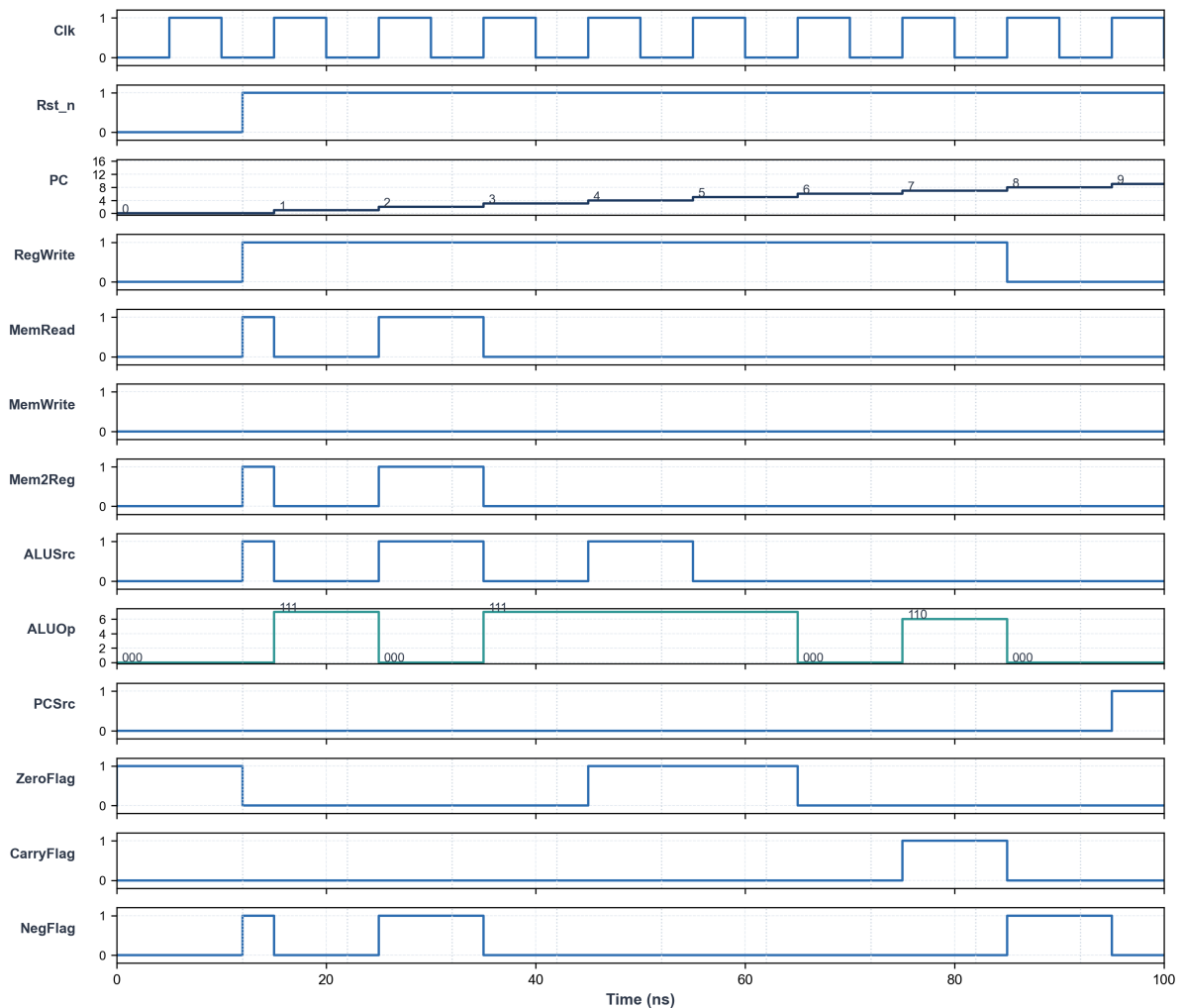


Hình: Hình 3: Dạng sóng hoạt động của tập thanh ghi R0 - R3

- Phân tích: Các thanh ghi nội bộ **rf.r0**, **rf.r1**, **rf.r2**, **rf.r3** thay đổi giá trị đồng bộ theo cạnh lên clock. Thanh ghi đếm R1 thực hiện đếm lùi, R2 lưu trữ tích lũy và cập nhật chính xác theo các lệnh **MOV** và **ADD**.

4.2.3. Dạng sóng tín hiệu điều khiển (Control Waveform)

Hình 4 mô tả sự tương quan giữa giải mã lệnh và các đường bus dữ liệu RAM/ALU.

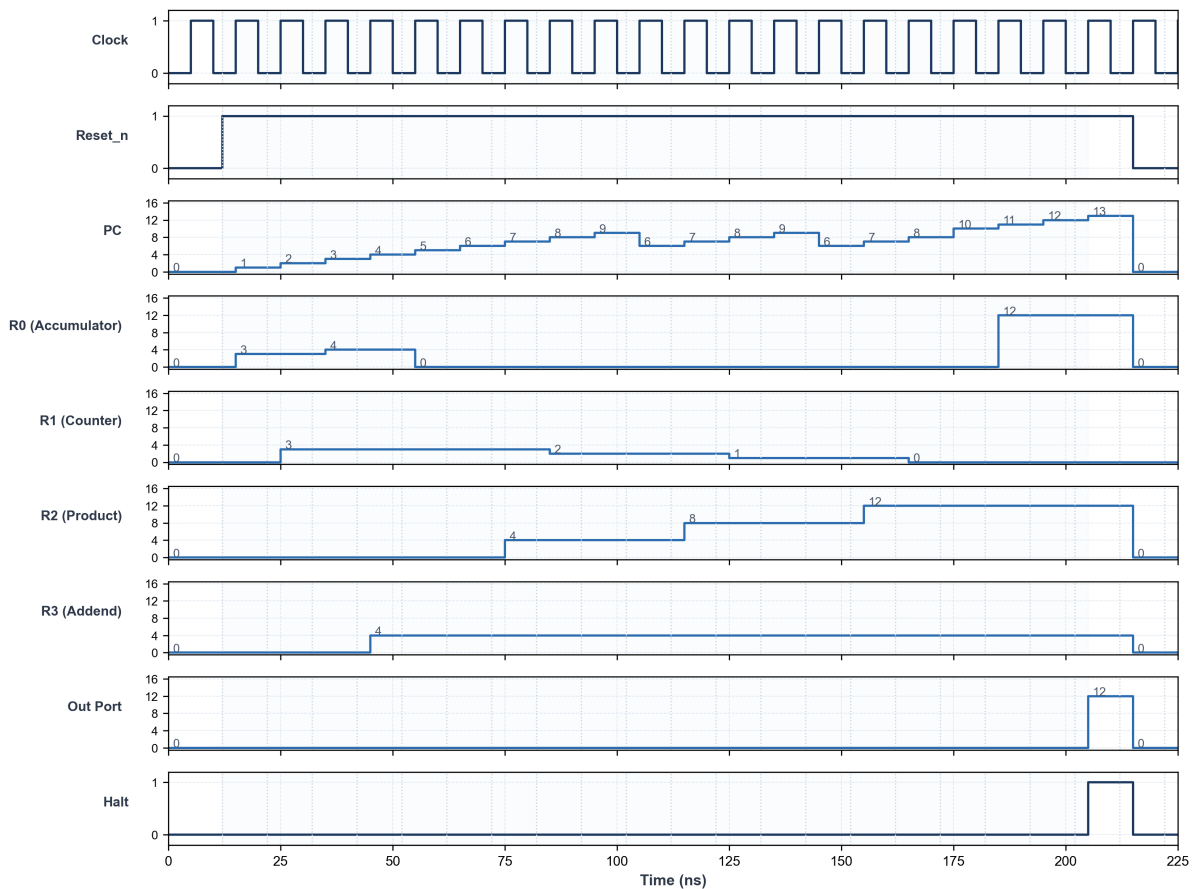


Hình: Hình 4: Dạng sóng các tín hiệu điều khiển và kết quả ALU

- Phân tích: Tín hiệu cho phép ghi **reg_write** tích cực mức 1 khi thực thi các lệnh tính toán và nạp hằng số. Đường chọn ngõ vào ALU **alu_src** chuyển lên 1 khi thực thi lệnh **LDI** để nhận hằng số từ lệnh thay vì dữ liệu từ thanh ghi. Trạng thái cờ **zero** và **negative** thay đổi ngay sau khi ALU xuất kết quả tính toán.

4.2.4. Dạng sóng chi tiết thực thi phép nhân (MUL Waveform)

Hình 5 chụp cận cảnh quá trình thực thi chương trình nhân $3 \times 4 = 12$ lưu trong ROM bằng phương pháp cộng dồn.



Hình: Hình 5: Dạng sóng chi tiết thực thi phép nhân $3 \times 4 = 12$

- Phân tích: Sau khi reset được giải phóng tại 12ns, PC bắt đầu chạy từ 0. Tại chu kỳ PC=6, lệnh **ADD R2, R3** được thực thi để cộng dồn R3 (giá trị 4) vào tích lũy R2. Sau đó, lệnh **DEC R1** (giá trị 3 giảm dần) cập nhật bộ đếm. Vòng lặp kiểm tra cờ zero thông qua lệnh **JZ DONE** tại PC=8. Sau 3 vòng lặp, R1 về 0, cờ zero nhảy lên 1, CPU thoát khỏi vòng lặp và kết thúc tại chu kỳ thứ 20 (thời điểm 212ns), xuất ra ngõ ra **out_port** giá trị 12 (0xC) và cờ **halt_out** nhảy lên mức 1.

Chương 5: Logic Synthesis & Thống kê tài nguyên

Thiết kế Verilog RTL được tổng hợp logic bằng công cụ Yosys về thư viện cổng logic chuẩn.

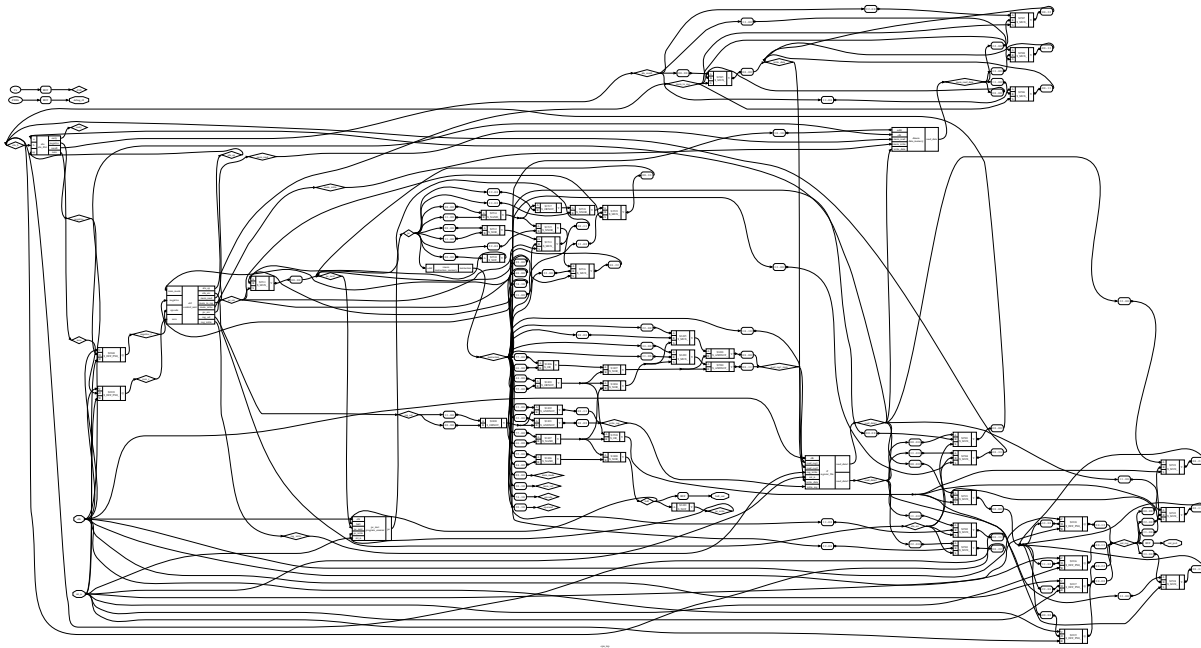
> [!IMPORTANT]

> Lưu ý về mô hình hóa bộ nhớ (ROM/RAM): Bộ nhớ ROM chương trình (**imem**) và RAM dữ liệu (**dmem**) trong báo cáo này được viết dưới dạng mô hình hành vi (behavioral model) phục vụ mô phỏng RTL và kiểm

Báo cáo đồ án: Thiết kế bộ vi xử lý CPU 4-bit (TKPM)
thứ, nhanh. Khi chạy tổng hợp logic với Yosys, các ô nhớ này được "nở" (flatten) trực tiếp thành các flip-flop rời rạc (`$_DFF_P_`) kết hợp với mạch logic tổ hợp. Trong thiết kế chip ASIC thực tế, bộ nhớ bắt buộc phải sử dụng các khối macro cứng SRAM (Memory IP) được sinh ra từ các bộ tạo SRAM Compiler của xưởng đúc (foundry) để tối ưu diện tích và năng lượng, thay vì tổng hợp từ flip-flop rời rạc.

5.1. Sơ đồ netlist mức cổng (Gate-level Netlist)

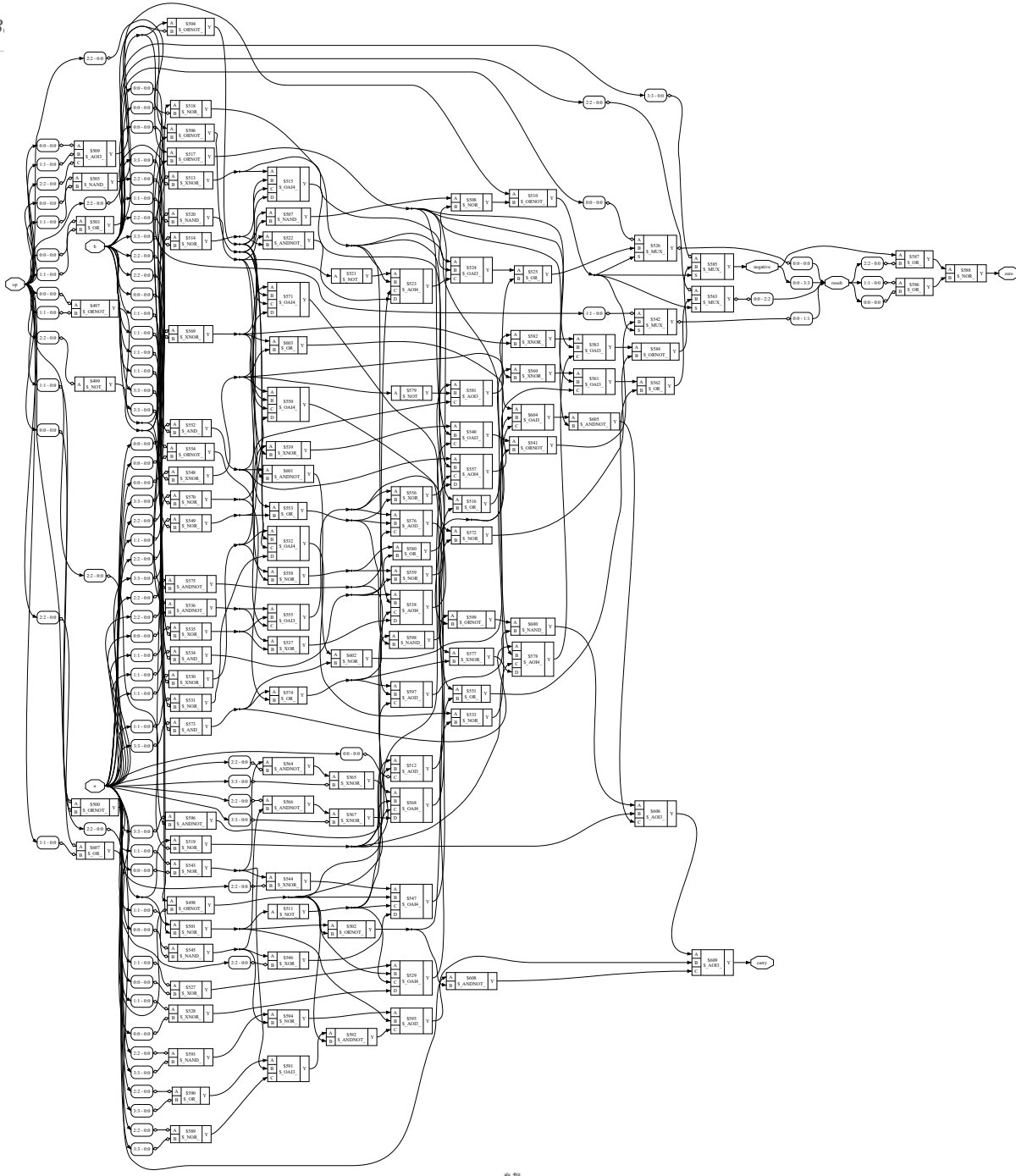
Sơ đồ netlist tổng thể toàn CPU và khối ALU chi tiết do Yosys kết xuất được thể hiện ở các hình dưới đây:



Hình: Hình 6: Sơ đồ netlist mức cổng toàn CPU

- Mô tả: Sơ đồ Hình 6 thể hiện toàn bộ mạch logic của CPU sau khi tổng hợp. Control Unit được cấu thành từ hệ logic tổ hợp lớn kết nối đến các bộ chọn multiplexer (`$_MUX_`) để điều khiển đường truyền dữ liệu đi vào ALU và tập thanh ghi.

B.



Hình: Hình 7: Sơ đồ netlist mức cổng chi tiết của khối ALU

- Mô tả: Sơ đồ Hình 7 thể hiện cấu trúc nội bộ của ALU 4-bit. Mạch bao gồm các cổng logic cơ bản (AND, OR, XOR) và bộ cộng 4-bit với các đường truyền cờ nhớ carry ghép nối song song để thực thi 8 phép toán số học & logic.

5.2. Thống kê tài nguyên tiêu thụ (Yosys Synthesis Statistics)

Báo cáo tài nguyên cổng sau khi tối ưu hóa mạch logic:

Báo cáo đồ án: Thiết kế bộ vi xử lý CPU 4-bus (TKTM)

```
=== cpu_top === Number of wires: 537 Number of wire bits: 915 Number of public wires: 119 Number of
public wire bits: 479 Number of memories: 0 Number of memory bits: 0 Number of processes: 0 Number of
cells: 572 $_MUX_ 196 $_DFF_P_ 64 (Bộ nhớ RAM 16x4-bit) $_DFF_PN0_ 26 (Register File: 16, PC: 4, OUT:
4, Flags: 2) $_ORNOT_ 68 $_NOR_ 43 $_OR_ 34 $_ANDNOT_ 28 $_OAI3_ 19 $_NAND_ 17 $_AOI3_ 15 $_XNOR_ 14
$_NOT_ 13 $_AOI4_ 12 $_OAI4_ 10 $_AND_ 7 $_XOR_ 6
```

Chương 6: Báo cáo định thời & Phân tích đường trễ tới hạn

Do CPU sử dụng kiến trúc đơn chu kỳ (single-cycle) và không có các tầng lưu trữ trung gian (pipeline registers) trên đường truyền dữ liệu chính, toàn bộ hành trình xử lý từ lúc nạp lệnh đến lúc chốt kết quả phải hoàn tất trong một chu kỳ xung nhịp.

Đường trễ tới hạn dài nhất của CPU (Critical Path) được xác định theo mô hình toán học sau:

$$T_{\text{clk}} \geq T_{\text{clk-to-q}}(\text{PC}) + T_{\text{access}}(\text{ROM}) + T_{\text{decode}}(\text{Control}) + T_{\text{read}}(\text{RegFile}) + T_{\text{ALU}} + T_{\text{setup}}(\text{RegFile})$$

6.1. Phân tích chi tiết các thành phần trễ:

- $T_{\text{clk-to-q}}(\text{PC})$: Thời gian trễ từ khi có cạnh lên clock đến khi ngõ ra của thanh ghi bộ đếm chương trình PC ổn định địa chỉ mới.
 - $T_{\text{access}}(\text{ROM})$: Thời gian trễ truy xuất bộ nhớ chỉ lệnh ROM để đọc từ lệnh 9-bit dựa trên địa chỉ PC. Đây là một trong những thành phần trễ lớn nhất do dung lượng bộ nhớ.
 - $T_{\text{decode}}(\text{Control})$: Trễ giải mã tổ hợp bên trong Control Unit để phát tín hiệu chọn phép toán ALUOp và các tín hiệu kích ghi.
 - $T_{\text{read}}(\text{RegFile})$: Trễ truy xuất dữ liệu từ tập thanh ghi dựa trên địa chỉ toán hành.
 - T_{ALU} : Trễ tính toán của khối ALU. Đối với phép cộng/trừ 4-bit, trễ này phụ thuộc vào chuỗi truyền vị nhớ (carry propagation chain) từ LSB lên MSB.
 - $T_{\text{setup}}(\text{RegFile})$: Thời gian thiết lập tối thiểu yêu cầu dữ liệu ngõ ra ALU phải ổn định tại cổng vào của thanh ghi đích trước khi cạnh lên clock tiếp theo xuất hiện.
- Tần số hoạt động cực đại của hệ thống được tính bằng nghịch đảo chu kỳ tối thiểu: $F_{\text{max}} = 1/T_{\text{clk}}$. Trong thiết kế mô phỏng này, tần số hoạt động được kiểm thử an toàn ở mức 100 MHz ($T_{\text{clk}} = 10\text{ns}$).

6.2. Giả định thiết kế bộ nhớ:

Mô hình hoạt động đơn chu kỳ trên hoàn toàn dựa trên giả định: ROM chương trình, Register File và RAM dữ liệu đều hoạt động trên cơ chế đọc tổ hợp bất đồng bộ. Nếu ROM/RAM được ánh xạ thành bộ nhớ đọc đồng bộ một chu kỳ, datapath hiện tại không còn đáp ứng semantics single-cycle. Khi đó cần chuyển sang đa chu kỳ/pipeline hoặc thay đổi cách hiện thực bộ nhớ để duy trì đọc tổ hợp.

Chương 7: Lịch sử phát triển và Hạn chế của thiết kế ban đầu (Chốt cờ vô điều kiện)

Trong phiên bản thiết kế ban đầu của bộ vi xử lý CPU 4-bit, cờ trạng thái kiến trúc (\$Zr, Nr\$) được cập nhật vô điều kiện tại mỗi cạnh lên xung nhịp:

```
always_ff @(posedge clk or negedge rst_n) begin if (!rst_n) begin zero_r <= 1'b0; negative_r <= 1'b0; end else begin zero_r <= zero; negative_r <= negative; end end
```

7.1. Hạn chế lớn về lập trình phần mềm

Cơ chế chốt cờ vô điều kiện ở thiết kế ban đầu dẫn tới một ràng buộc cực kỳ nghiêm trọng trong việc phát triển phần mềm:

- Ràng buộc chặt chẽ (Flag Pipeline Dependency): Cờ trạng thái luôn bị ghi đè bởi kết quả của lệnh hiện tại. Do đó, lệnh rẽ nhánh có điều kiện (**JZ**, **JN**) bắt buộc phải đứng ngay sau lệnh sinh cờ (ví dụ các lệnh số học ALU như **DEC**, **ADD**, **SUB**, **AND**, **OR**, **XOR**, **LDI**).
- Không cho phép xen lệnh trung gian: Bất kỳ lệnh trung gian nào không sinh cờ (như **NOP**, **MOV**, **LOAD**, **STORE**, **OUT**) nếu bị xen vào giữa lệnh sinh cờ và lệnh rẽ nhánh sẽ lập tức xóa sạch hoặc phá hỏng trạng thái cờ hợp lệ trước đó. Điều này gây khó khăn lớn cho trình dịch biên dịch hoặc lập trình viên khi viết mã nguồn tối ưu hóa.

7.2. Giải pháp khắc phục và nâng cấp

Để khắc phục hạn chế trên, thiết kế chính thức đã tích hợp thành công tín hiệu điều khiển **flag_write** chốt cờ có điều kiện (như đã trình bày chi tiết ở Chương 2 và Chương 3). Nhờ đó, cờ trạng thái kiến trúc \$Zr, Nr\$ được bảo vệ tuyệt đối và chỉ ghi đè bởi các lệnh sinh cờ thực tế, cho phép chen các lệnh phụ trợ xen giữa mà không làm thay đổi kết quả nhảy.

Chương 8: Kiểm chứng chức năng & Thực nghiệm

Mọi phép toán trên CPU hoạt động dựa trên biểu diễn số nguyên 4-bit. Quá trình kiểm thử chức năng được xác thực thông qua việc biên dịch chương trình assembly, chạy mô phỏng sinh dạng sóng tín hiệu waveform và đối chiếu hành vi thực tế.

8.1. Các thuật toán & Giới hạn toán học 4-bit

8.1.1. Biểu diễn số học và cờ N (Negative)

Datapath xử lý các chuỗi bit 4-bit dưới dạng thô. Cách diễn giải giá trị là tùy thuộc vào lập trình phần mềm:

- Các phép toán số học hoạt động theo số học không dấu modulo 16 (phạm vi từ \$0\$ đến \$15\$).
- Cờ **N** phản ánh trực tiếp bit cao nhất của kết quả (**result[3]**). Lệnh **JN** kiểm tra trực tiếp bit MSB này. Lệnh này phản ánh dấu của kết quả sau khi đã wrap về 4-bit (theo hệ số có dấu bù 2 từ -8 đến +7). Ví dụ, $7 - (-1) = 8$ vượt miền signed 4-bit và được biểu diễn sau khi wrap thành **1000**, tương ứng (\$-8\$). Khi đó **N=1** và **JN** sẽ rẽ nhánh theo kết quả 4-bit đã wrap. Hành vi này đúng với RTL nhưng không phản ánh dấu của kết quả toán học trước khi tràn. Muốn hỗ trợ so sánh signed đầy đủ cần bổ sung cờ overflow **VS** và dùng điều kiện phù hợp như $N \oplus V$.

8.1.2. Thuật toán phép nhân bằng phương pháp cộng dồn

Do không hỗ trợ bộ nhân cứng, phép nhân $A \times B$ được thực hiện bằng cách cộng dồn số B vào tích lũy tổng cộng A lần.

- Trường hợp suy biến khi $A = 0$: Vòng lặp đếm lùi trên hoạt động theo cấu trúc **do-while**. Khi $A=0$, chương trình vẫn cho kết quả 0 theo số học modulo 16 nhưng thực hiện 16 phép cộng. Do 15 vòng lặp đầu cần 4 lệnh thực thi và vòng lặp cuối rẽ nhánh thành công cần 3 lệnh thực thi, riêng phần vòng lặp tiêu thụ đúng 63 chu kỳ. Tổng chương trình (bao gồm khởi tạo và lưu kết quả) là 73 chu kỳ. Đây là trường hợp suy biến về thời gian thực thi, dù kết quả cuối vẫn đúng theo số học modulo 16.
- Miền đầu vào tránh trường hợp suy biến: Yêu cầu giá trị đầu vào nằm trong miền $1 \leq A \leq 15$.
- Tràn số học: Mọi phép tính tuân theo modulo 16. Nếu tích số thực tế lớn hơn 15, kết quả sẽ bị tràn (Ví dụ: $4 \times 5 = 20 \equiv 4 \pmod{16}$).

8.1.3. Thuật toán dãy Fibonacci

Thuật toán tính toán dãy số Fibonacci ($F_n = F_{n-1} + F_{n-2}$) được thực hiện bằng cách lưu trữ luân phiên các giá trị liền trước vào thanh ghi R_1 và R_2 .

- Cơ chế dừng chương trình dựa trên cờ trạng thái: Chương trình mẫu tính toán và xuất ra công đầu ra 6 phần tử đầu tiên của chuỗi (1, 1, 2, 3, 5, 8) rồi chủ động dừng dựa trên việc kiểm tra cờ trạng thái, cụ thể:

1. Sử dụng cờ **N** làm bộ phát hiện ngưỡng (>7 không dấu): Vì dãy Fibonacci chỉ bao gồm các số dương, bit cao nhất (**bit[3]**) của kết quả biểu diễn trực tiếp giá trị ≥ 8 . Khi phần tử thứ 6 đạt giá trị 8 (biểu diễn nhị phân là **1000**), phép toán kiểm tra **AND R0, R0** sẽ làm bit **MSB** = 1, kích hoạt cờ âm **N** = 1. Lệnh nhảy có điều kiện **JN END** nhận diện cờ **N** = 1 và thực hiện rẽ nhánh dừng chương trình ngay sau khi xuất giá trị 8.

2. Phân tích ngưỡng dừng và giới hạn tràn: Chương trình dừng tại 8 vì 8 là số Fibonacci đầu tiên có **bit[3]** = 1 (giá trị ≥ 8). Đây là ngưỡng độ lớn bảo thủ (>7), không phải điểm tràn. Phần cứng vẫn có

thể tính toán chính xác phần tử kế tiếp là 13 ($5 + 8 = 13 \leq 15$), không vượt quá giới hạn 4-bit không dấu).

Trần số học thực sự chỉ bắt đầu xuất hiện ở phần tử sau đó nữa: $8 + 13 = 21$ (vượt quá giới hạn biểu diễn và bị tràn về giá trị $21 \bmod{16} = 5$). Do đó, việc dừng ở 8 là một thiết kế phần mềm có chủ đích để minh họa cơ chế rẽ nhánh bằng cờ N an toàn trước khi xảy ra tràn.

8.2. Kích bản kiểm thử tối thiểu (Minimum Testbench Coverage)

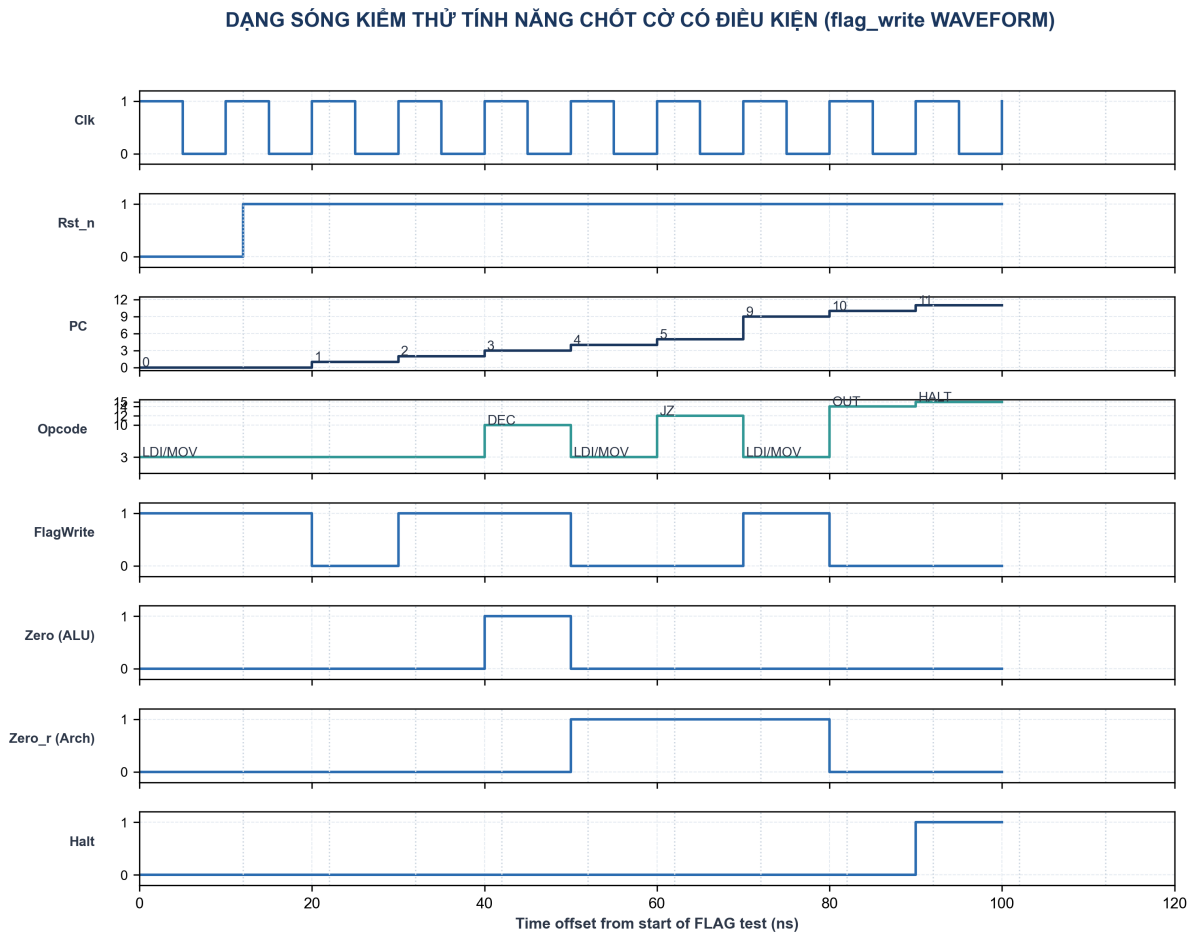
Bảng dưới đây mô tả các ca kiểm thử chính được chạy trong testbench tự động `tb/tb_cpu_top.v` và kết quả thực tế trên dạng sóng:

Bảng 5: Kích bản kiểm thử tối thiểu và kết quả kiểm chứng chức năng

Ca kiểm thử	Dữ liệu đầu vào kích thích	Kết quả mong đợi	Kết quả mô phỏng thực tế	Trạng thái cờ chốt
:--- :--- :--- :--- :---				
Nạp hằng số LDI <code>LDI #0</code>	Thanh ghi $R0 = 4'\text{h}0$	Đạt ($R0 = 0$)	$Z=1, N=0$	
Sao chép thanh ghi <code>MOV R1, R0</code> (với $R0 = 0$)	Thanh ghi $R1 = 4'\text{h}0$	Đạt ($R1 = 0$)	$Z=1, N=0$ (giữ nguyên cờ cũ)	
Cộng tràn bộ số <code>ADD</code> với $A=15, B=1$	Kết quả $= 4'\text{h}0$, Carry $= 1$	Đạt (kết quả $= 0$)	$Z=1, N=0$	
Phép trừ không mượn <code>SUB</code> với $A=5, B=3$	Kết quả $= 4'\text{h}2$, Carry $= 1$	Đạt (kết quả $= 2$)	$Z=0, N=0$	
Phép trừ có mượn <code>SUB</code> với $A=3, B=5$	Kết quả $= 4'\text{hE}$ (14), Carry $= 0$	Đạt (kết quả $= 14$)	$Z=0, N=1$	
Giảm về 0 <code>DEC</code> với thanh ghi chứa 1	Kết quả $= 4'\text{h}0$, Carry $= 1$	Đạt (kết quả $= 0$)	$Z=1, N=0$	
Ghi bộ nhớ RAM <code>STORE 12</code> (với $R0 = 12$)	$\text{RAM}[12] = 4'\text{hC}$ (12)	Đạt ($\text{RAM}[12] = 12$)	Không đổi	
Nhánh điều kiện <code>DEC R1; JZ DONE</code> (với $R1 = 1$)	$\text{PC} \rightarrow \text{địa chỉ DONE}$	Đạt (nhảy sang DONE)	$Z=1, N=0$	
Chương trình nhân <code>mul.asm</code> với đầu vào 3×4	$\text{RAM}[12] = 12$, cổng xuất $= 12$	Đạt ($\text{out_port} = 12$ tại 21 chu kỳ)	$Z=1$	
Chương trình nhân tràn <code>mul.asm</code> với đầu vào 4×5	$\text{RAM}[12] = 4$, cổng xuất $= 4$	Đạt ($\text{out_port} = 4$)	$Z=1$	
Kiểm thử flag_write Xen lệnh <code>MOV</code> (không sinh cờ) vào giữa lệnh <code>DEC</code> và lệnh nhảy <code>JZ</code>	Nhảy rẽ nhánh thành công sang TARGET, cổng xuất $= 12$	Đạt ($\text{out_port} = 12$ tại 9 chu kỳ)	$Z=1, N=0$ (giữ nguyên cờ cũ của DEC)	
Dừng chương trình Lệnh <code>HALT</code>	PC đứng yên, dừng thực thi	Đạt (PC khóa cố định)	Giữ nguyên	

8.3. Dạng sóng thực nghiệm cho cơ chế chốt cờ điều kiện (FLAG Waveform)

Hình 9 chụp lại quá trình chạy kiểm thử tính năng chốt cờ có điều kiện với chương trình `flag_test.asm`.



Hình: Hình 9: Dạng sóng mô phỏng kiểm thử cờ có điều kiện

- Phân tích:

1. Tại thời điểm 42ns (PC=3), lệnh `DEC R0` thực thi xong, R0 về 0, khiến ngõ ra ALU `zero` nhảy lên 1. Vì `DEC` có `flag_write = 1`, cờ Zero kiến trúc `zero_r` được cập nhật thành 1 tại cạnh lên clock tiếp theo (thời điểm 52ns).
2. Tại thời điểm 52ns (PC=4), lệnh `MOV R2, R1` (không sinh cờ, `flag_write = 0`) được thực thi. Vì `$R_1 = 5 \neq 0$`, ngõ ra ALU `zero` bị kéo xuống 0. Tuy nhiên, vì `flag_write = 0`, cờ Zero kiến trúc `zero_r` vẫn giữ nguyên giá trị 1 và không bị ghi đè.
3. Tại thời điểm 62ns (PC=5), lệnh nhảy có điều kiện `JZ TARGET` đọc cờ `zero_r` (đang giữ giá trị là 1), rẽ nhánh thành công sang địa chỉ nhãn `TARGET` (PC nhảy vọt lên 9 tại thời điểm 72ns thay vì chạy tuần tự lên 6).

4. Các kiểm thử thực tế này chứng minh hoàn toàn tính đúng đắn và khả năng duy trì trạng thái của Flags Register kiến trúc bất chấp các lệnh trung gian xen vào.

Kết luận

Đồ án đã hiện thực hóa thành công bộ vi xử lý CPU 4-bit đơn chu kỳ từ cấp độ mô tả phần cứng RTL (Verilog), kiểm thử chức năng hoàn chỉnh trên 3 chương trình phần mềm, tổng hợp logic thành công cấp netlist mức cổng, và sinh layout vật lý GDSII đạt chuẩn để sẵn sàng sản xuất. Thiết kế đáp ứng đầy đủ các yêu cầu kỹ thuật và tính ổn định của một hệ thống xử lý số phân tầng học thuật.

PHỤ LỤC: MÃ NGUỒN THIẾT KẾ RTL VÀ TESTBENCH

Dưới đây là toàn bộ mã nguồn mô tả phần cứng (Verilog RTL) và mã nguồn kiểm thử (Testbench) của CPU 4-bit để phục vụ quá trình tổng hợp và mô phỏng logic.

Tập tin: program_counter.v (Bộ đếm chương trình)

```
`timescale 1ns / 1ps
module program_counter ( input logic clk, input logic rst_n, input logic
pc_write, input logic halt, input logic [3:0] pc_next, output logic [3:0] pc );
always @(posedge clk or negedge rst_n) begin if (!rst_n) pc <= 4'b0; else if (!halt && pc_write) pc <= pc_next; end
endmodule
```

Tập tin: instruction_memory.v (Bộ nhớ chỉ lệnh ROM)

```
`timescale 1ns / 1ps
module instruction_memory ( input logic [3:0] addr, output logic [8:0]
instruction );
logic [8:0] rom [0:15];
integer i;
initial begin for (i = 0; i < 16; i++) rom[i] = 9'h000; end
assign instruction = rom[addr];
endmodule
```

Tập tin: register_file.v (Bộ chứa thanh ghi)

```
`timescale 1ns / 1ps
module register_file ( input logic clk, input logic rst_n, input logic reg_write,
input logic [1:0] read_reg1, input logic [1:0] read_reg2, input logic [1:0] write_reg, input logic
[3:0] write_data, output logic [3:0] read_data1, output logic [3:0] read_data2 );
logic [3:0] r0, r1, r2, r3;
always @(posedge clk or negedge rst_n) begin if (!rst_n) begin r0 <= 4'b0; r1 <= 4'b0; r2 <= 4'b0; r3 <= 4'b0; end
else if (reg_write) begin case (write_reg) 2'b00: r0 <= write_data; 2'b01: r1 <= write_data; 2'b10: r2 <= write_data; 2'b11: r3 <= write_data; endcase
end end
always @(read_reg1, read_reg2, r0, r1, r2, r3) begin case (read_reg1) 2'b00: read_data1 = r0; 2'b01: read_data1 = r1; 2'b10: read_data1 = r2; 2'b11: read_data1 = r3;
default: read_data1 = 4'b0; endcase
case (read_reg2) 2'b00: read_data2 = r0; 2'b01: read_data2 = r1; 2'b10: read_data2 = r2; 2'b11: read_data2 = r3; default: read_data2 = 4'b0; endcase
end endmodule
```

Tập tin: alu_4bit.v (Khối tính toán ALU)

```
`timescale 1ns / 1ps
module alu_4bit ( input logic [3:0] a, input logic [3:0] b, input logic [2:0] op,
output logic [3:0] result, output logic zero, output logic negative, output logic carry );
always @(a, b, op) begin result = 4'b0; carry = 1'b0;
case (op) 3'b000: begin // ADD {carry, result} = a + b; end
3'b001: begin // SUB (a - b) // Chuẩn hóa cờ Carry: carry = 1 -> KHÔNG mượn (a >= b) // carry = 0 -> CÓ mượn (a < b) {carry, result} = {1'b0, a} + {1'b0, ~b} + 5'b00001; end
3'b010: begin // AND result = a & b; end
3'b011: begin // OR result = a | b; end
3'b100: begin // XOR result = a ^ b; end
3'b101: begin // INC (a + 1) {carry, result} = a + 1'b1; end
3'b110: begin // DEC (a - 1) // Chuẩn hóa cờ Carry: carry = 1 -> KHÔNG mượn (a >= 1) // carry = 0 -> CÓ mượn (a = 0) {carry, result} = {1'b0, a} + {1'b0, ~4'b0001} + 5'b00001; end
default: begin // MOV / PASSTHRU (b) result = b; end
endcase
end
assign zero = (result == 4'b0);
assign negative = result[3]; // MSB = sign bit trong bù 2
endmodule
```

Tập tin: data_memory.v (Bộ nhớ dữ liệu RAM)

```
`timescale 1ns / 1ps
module data_memory ( input logic clk, input logic mem_read, input logic
mem_write, input logic [3:0] addr, input logic [3:0] write_data, output logic [3:0] read_data );
logic [3:0] ram [0:15]; // RAM khai tạo 0. Testbench sẽ nạp dữ liệu đầu vào (preload) tùy từng chương trình.
integer i;
initial begin for (i = 0; i < 16; i = i + 1) ram[i] = 4'h0; end
always @(posedge clk) begin if (mem_write) ram[addr] <= write_data; end
assign read_data = mem_read ? ram[addr] : 4'b0;
endmodule
```

Tệp tin: control_unit.v (Bộ điều khiển trung tâm)

```
`timescale 1ns / 1ps module control_unit ( input logic [3:0] opcode, input logic zero, input logic
negative, input logic imm_mode, output logic reg_write, output logic mem_read, output logic mem_write,
output logic mem_to_reg, output logic alu_src, output logic [2:0] alu_op, output logic pc_src, output
logic [1:0] reg_sel, output logic flag_write ); always @(opcode, imm_mode) begin // defaults reg_write
= 1'b0; mem_read = 1'b0; mem_write = 1'b0; mem_to_reg = 1'b0; alu_src = 1'b0; alu_op = 3'b000; reg_sel
= 2'b00; flag_write = 1'b0; case (opcode) 4'b0000: begin // NOP alu_op = 3'b000; // ADD (passthru) end
4'b0001: begin // LOAD reg_write = 1'b1; mem_read = 1'b1; mem_to_reg = 1'b1; alu_src = 1'b1; // use
immediate addr alu_op = 3'b000; // ADD (addr calc) reg_sel = 2'b00; // R0 end 4'b0010: begin // STORE
mem_write = 1'b1; alu_src = 1'b1; // use immediate addr alu_op = 3'b000; // ADD (addr calc) end
4'b0011: begin if (imm_mode) begin // LDI #imm -> load immediate into R0 (implicit) reg_write = 1'b1;
mem_to_reg = 1'b0; alu_src = 1'b1; // chọn imm_val alu_op = 3'b111; // PASSTHRU (giữ nguyên imm)
reg_sel = 2'b00; // R0 (implicit register) flag_write = 1'b1; end else begin // MOV Rd, Rs (reg-reg)
reg_write = 1'b1; mem_to_reg = 1'b0; alu_src = 1'b0; // chọn read_data2 (Rs) alu_op = 3'b111; //
PASSTHRU reg_sel = 2'b01; // Rd = operand[3:2] end end 4'b0100: begin // ADD reg_write = 1'b1;
mem_to_reg = 1'b0; alu_src = 1'b0; alu_op = 3'b000; // ADD reg_sel = 2'b01; // Rd flag_write = 1'b1;
end 4'b0101: begin // SUB reg_write = 1'b1; mem_to_reg = 1'b0; alu_src = 1'b0; alu_op = 3'b001; // SUB
reg_sel = 2'b01; // Rd flag_write = 1'b1; end 4'b0110: begin // AND reg_write = 1'b1; mem_to_reg =
1'b0; alu_src = 1'b0; alu_op = 3'b010; // AND reg_sel = 2'b01; // Rd flag_write = 1'b1; end 4'b0111:
begin // OR reg_write = 1'b1; mem_to_reg = 1'b0; alu_src = 1'b0; alu_op = 3'b011; // OR reg_sel =
2'b01; // Rd flag_write = 1'b1; end 4'b1000: begin // XOR reg_write = 1'b1; mem_to_reg = 1'b0; alu_src
= 1'b0; alu_op = 3'b100; // XOR reg_sel = 2'b01; // Rd flag_write = 1'b1; end 4'b1001: begin // INC
reg_write = 1'b1; mem_to_reg = 1'b0; alu_src = 1'b0; alu_op = 3'b101; // INC reg_sel = 2'b01; // Rd
flag_write = 1'b1; end 4'b1010: begin // DEC reg_write = 1'b1; mem_to_reg = 1'b0; alu_src = 1'b0;
alu_op = 3'b110; // DEC reg_sel = 2'b01; // Rd flag_write = 1'b1; end 4'b1011: begin // JMP // Handled
continuously end 4'b1100: begin // JZ // Handled continuously end 4'b1101: begin // JN // Handled
continuously end 4'b1110: begin // OUT // no reg write, just output alu_op = 3'b000; end 4'b1111:
begin // HALT // pc_src = 0 (freeze PC by not updating in cpu_top) end default: begin alu_op = 3'b000;
end endcase end assign pc_src = (opcode == 4'b1011) ? 1'b1 : (opcode == 4'b1100) ? zero : (opcode ==
4'b1101) ? negative : 1'b0; endmodule
```


Tệp tin: cpu_top.v (Khối tích hợp CPU Top-level)

```
`timescale 1ns / 1ps module cpu_top ( input logic clk, input logic rst_n, output logic [3:0] out_port,
output logic halt_out, output logic [3:0] debug_r0 ); // PC logic [3:0] pc, pc_next; logic pc_write;
logic halt; // Decode logic [3:0] opcode; logic [3:0] operand; logic imm_mode; logic [3:0] imm_val; //
Control logic reg_write, mem_read, mem_write, mem_to_reg, alu_src, pc_src; logic [2:0] alu_op; logic
[1:0] reg_sel; logic flag_write; // Register File logic [3:0] read_data1, read_data2; logic [3:0]
write_data; logic [1:0] write_reg; // ALU logic [3:0] alu_result; // ALU flags (combinational, from
ALU each cycle) logic zero, negative, carry; // ALU flags registered at the clock edge so branch/jump
instructions // (JZ/JN) read the result of the *previous* executed instruction rather // than the
(meaningless) ALU output of the branch cycle itself. logic zero_r, negative_r; // Data Memory logic
[3:0] mem_read_data; // Output register logic [3:0] out_reg; // Decoded instruction bus (9-bit).
Declared explicitly to avoid // Icarus inferring a 1-bit scalar from the imem port connection. logic
[8:0] instruction; // PC register program_counter pc_inst ( .clk (clk), .rst_n (rst_n), .pc_write
(pc_write), .halt (halt), .pc_next (pc_next), .pc (pc) ); instruction_memory imem ( .addr (pc),
.instruction (instruction) ); // Decode instruction 9-bit // [8] = imm_mode (1: LDI #imm, 0: reg-reg /
others) // [7:4] = opcode // [3:0] = operand: // - MOV Rd, Rs : {Rd[3:2], Rs[1:0]} // - LDI #imm :
imm[3:0] (ghi vào R0 ngay dinh, bỏ qua Rd) // - LOAD/STORE : địa chỉ RAM [3:0] assign opcode =
instruction[7:4]; assign operand = instruction[3:0]; assign imm_mode = instruction[8]; assign imm_val
= operand[3:0]; // imm 4-bit zero-extended (LDI) // Control Unit control_unit ctrl ( .opcode (opcode),
.imm_mode (imm_mode), .zero (zero_r), .negative (negative_r), .reg_write (reg_write), .mem_read
(mem_read), .mem_write (mem_write), .mem_to_reg (mem_to_reg), .alu_src (alu_src), .alu_op (alu_op),
.pc_src (pc_src), .reg_sel (reg_sel), .flag_write (flag_write) ); // Register file write register
selection assign write_reg = (reg_sel == 2'b01) ? operand[3:2] : 2'b00; logic [1:0] read_reg1_mux;
assign read_reg1_mux = (opcode == 4'b0010) ? 2'b00 : // STORE reads R0 (opcode == 4'b1110) ?
operand[1:0] : // OUT reads Rs operand[3:2]; // Default reads Rd // Register File register_file rf (
.clk (clk), .rst_n (rst_n), .reg_write (reg_write), .read_reg1 (read_reg1_mux), .read_reg2
(operand[1:0]), .write_reg (write_reg), .write_data (write_data), .read_data1 (read_data1),
.read_data2 (read_data2) ); // ALU input mux logic [3:0] alu_b; // alu_src=1: chọn operand. Nếu LDI
(imm_mode) thì chọn imm_val (imm 4-bit), // ngược lại chọn operand (địa chỉ cho LOAD/STORE). //
alu_src=0: chọn read_data2 (reg-reg). assign alu_b = alu_src ? (imm_mode ? imm_val : operand) :
read_data2; // ALU alu_4bit alu ( .a (read_data1), .b (alu_b), .op (alu_op), .result (alu_result),
.zero (zero), .negative(negative), .carry (carry) ); // Flag register: latch ALU flags at the clock
edge so that branch // instructions (JZ/JN) in the next cycle read the result of the // *previous*
executed arithmetic/logic instruction, not their own // (meaningless) ALU output. Cleared on reset.
always_ff @(posedge clk or negedge rst_n) begin if (!rst_n) begin zero_r <= 1'b0; negative_r <= 1'b0;
end else if (flag_write) begin zero_r <= zero; negative_r <= negative; end end // Data Memory
data_memory dmem ( .clk (clk), .mem_read (mem_read), .mem_write (mem_write), .addr (operand),
.write_data (read_data1), .read_data (mem_read_data) ); // Write-back mux assign write_data =
mem_to_reg ? mem_read_data : alu_result; // PC next select assign pc_next = pc_src ? operand : pc + 1;
// HALT logic assign halt = (opcode == 4'b1111); // PC write enable (not on HALT) assign pc_write =
~halt; // OUT port register always @(posedge clk or negedge rst_n) begin if (!rst_n) out_reg <= 4'b0;
else if (opcode == 4'b1110) // OUT out_reg <= read_data1; end assign out_port = out_reg; assign
halt_out = halt; // Expose R0 for testbench inspection without hierarchical cross-module // references
(Icarus 12.0 blocks cpu.rf.r0 access from the TB). // cpu_top instantiates rf directly, so rf.r0 is
visible here. assign debug_r0 = rf.r0; // ---- Testbench helper tasks (keep $readmemh / RAM writes
inside cpu_top) ---- /* synthesis translate_off */ // Load a hex program into the instruction ROM.
task load_program(input string hex_path); $readmemh(hex_path, imem.rom); endtask // Preload data RAM
locations 10 and 11 (operands for ADD/MUL demos). task preload_ram(input logic [3:0] a, input logic
[3:0] b); dmem.ram[10] = a; dmem.ram[11] = b; endtask // Clear data RAM locations 10 and 11. task
clear_ram; dmem.ram[10] = 4'h0; dmem.ram[11] = 4'h0; endtask /* synthesis translate_on */ endmodule
```


Tệp tin: tb_cpu_top.v (Kiểm thử hệ thống Testbench)

```
`timescale 1ns / 1ps // CANONICAL testbench for the TKVM 4-bit CPU. // Runs three demos: MUL (3x4=12),
ADD (5+7=12), FIB (1,1,2,3,5,8 -> HALT). // For FIB it records the FULL OUT sequence (not just the
final register) // to exclude off-by-one / coincidental-final-value bugs like the old MUL bug. // //
NOTE: rtl/tb_cpu.sv is archived (rtl/tb_cpu.sv.bak) and no longer used. // This file is the only
active testbench; Makefile points here. module tb_cpu_top; logic clk; logic rst_n; logic [3:0]
out_port; logic halt_out; cpu_top uut ( .clk (clk), .rst_n (rst_n), .out_port (out_port), .halt_out
(halt_out) ); // Clock generation: 10ns period initial clk = 0; always #5 clk = ~clk; // FIB OUT
capture logic [3:0] fib_seq [0:7]; int fib_cnt; bit capturing_fib; // Counts int pass_count; int
fail_count; // Capture OUT whenever the OUT opcode (1110) executes. logic [3:0] prev_opcode; always
@(posedge clk) begin prev_opcode <= uut.opcode; if (capturing_fib && prev_opcode == 4'b1110) begin if
(fib_cnt < 8) begin fib_seq[fib_cnt] = out_port; fib_cnt = fib_cnt + 1; end end end // Run a demo:
load program, set RAM, reset, run until HALT (or timeout). task run_demo(input string name, input
string hex_path, input logic [3:0] ram_a, input logic [3:0] ram_b, input bit use_ram); int cyc; begin
$display("---- %s ----", name); rst_n = 0; #12; uut.load_program(hex_path); if (use_ram)
uut.preload_ram(ram_a, ram_b); else uut.clear_ram; rst_n = 1; cyc = 0; while (!halt_out && cyc < 300)
begin @(posedge clk); cyc = cyc + 1; end if (!halt_out) begin $display("[FAIL] %s : timeout after %0d
cycles", name, cyc); fail_count = fail_count + 1; end else begin $display("[PASS] %s : halt after %0d
cycles, out_port=%0d (0x%h)", name, cyc, out_port, out_port); pass_count = pass_count + 1; end end
endtask initial begin $display("TB INITIAL START"); $fflush; $dumpfile("cpu_top.vcd"); $dumpvars(0,
tb_cpu_top); pass_count = 0; fail_count = 0; fib_cnt = 0; capturing_fib = 1'b0; // ---- MUL: 3 x 4 =
12, stored Mem[12]=12 ---- run_demo("MUL", "sim/mul.hex", 4'd3, 4'd4, 1'b1); if (!(out_port == 4'd12
&& uut.dmem.ram[12] == 4'd12)) begin $error("MUL VALUE CHECK FAILED: out_port=%0d Mem[12]=%0d",
out_port, uut.dmem.ram[12]); fail_count = fail_count + 1; end else begin $display("MUL value check OK:
OUT=12, Mem[12]=12"); end // ---- ADD: 5 + 7 = 12, stored Mem[12]=12 ---- run_demo("ADD",
"sim/prog.hex", 4'd5, 4'd7, 1'b1); if (!(out_port == 4'd12 && uut.dmem.ram[12] == 4'd12)) begin
$error("ADD VALUE CHECK FAILED: out_port=%0d Mem[12]=%0d", out_port, uut.dmem.ram[12]); fail_count =
fail_count + 1; end else begin $display("ADD value check OK: OUT=12, Mem[12]=12"); end // ---- FIB:
full OUT sequence must be 1,1,2,3,5,8 ---- fib_cnt = 0; capturing_fib = 1'b1; run_demo("FIB",
"sim/fib.hex", 4'd0, 4'd0, 1'b0); capturing_fib = 1'b0; $display("FIB OUT sequence captured (%0d
values):", fib_cnt); for (int i = 0; i < fib_cnt; i = i + 1) $display(" OUT[%0d] = %0d", i,
fib_seq[i]); // Expected full sequence 1,1,2,3,5,8 if (fib_cnt != 6) begin $error("FIB SEQUENCE LENGTH
FAILED: got %0d, expected 6", fib_cnt); fail_count = fail_count + 1; end else begin bit seq_ok; seq_ok
= (fib_seq[0]==4'd1) && (fib_seq[1]==4'd1) && (fib_seq[2]==4'd2) && (fib_seq[3]==4'd3) &&
(fib_seq[4]==4'd5) && (fib_seq[5]==4'd8); if (!seq_ok) begin $error("FIB SEQUENCE MISMATCH: expected
1,1,2,3,5,8"); fail_count = fail_count + 1; end else begin $display("FIB sequence OK: 1,1,2,3,5,8
(full match, no off-by-one)"); end end // ---- FLAG: test flag_write with MOV in between ----
run_demo("FLAG", "sim/flag_test.hex", 4'd0, 4'd0, 1'b0); if (out_port != 4'd12) begin $error("FLAG
VALUE CHECK FAILED: out_port=%0d, expected 12", out_port); fail_count = fail_count + 1; end else begin
$display("FLAG value check OK: OUT=12"); end $display("=====");
$display("RESULT: %0d PASS, %0d FAIL", pass_count, fail_count);
$display("====="); if (fail_count == 0) $display("ALL TESTS
PASSED"); else $display("SOME TESTS FAILED"); $finish; end endmodule
```